INGENIERÍA DE SOFTWARE I · CONTROL DE LECTURA

La app de Doña Rosa

Una sola historia, de principio a fin, que recorre todo lo que entra al control: SWEBOK, Scrum (Rubin), historias de usuario (Cohn y Leffingwell), cómo escribir requisitos (Wiegers) y Robertson.

Léela de corrido, como un cuento. Los recuadros son lo que escribes en el control:
naranja = SWEBOK · **azul** = libros (Rubin, Cohn, Leffingwell, Wiegers, Robertson).
«p. 46» = página del visor PDF del archivo en *material/*. Entre paréntesis, la página impresa del libro.
Lecturas según los anuncios de Canvas (2026-10-07 y 2026-10-12).

PRÓLOGO · SWEBOK · ROBERTSON CAP. 1

El sobrino

Qué es el software, qué es la ingeniería de software y por qué los requisitos importan.

Doña Rosa tiene una panadería frente a la ESPOL. Hace dos años su sobrino le hizo «una app» de pedidos en un fin de semana. Funcionó tres días. Al cuarto cobró dos veces a todos los clientes, nadie supo arreglarla y el sobrino se fue a vivir a España.

Ahora Doña Rosa te contrata a ti. Y tú, que ya sabes cómo terminó lo del sobrino, decides hacerlo distinto.

> **SWEBOK · SOFTWARE**
> Programas de computadora junto con sus procedimientos y, posiblemente, la documentación y los datos asociados a la operación del sistema.
> SWEBOK p. 38, Introducción

El sobrino dejó código y un pasaje de avión. Ni documentación, ni datos ordenados. Eso no es software completo. Y lo que hizo tampoco fue ingeniería:

> **SWEBOK · INGENIERÍA DE SOFTWARE (PREGUNTA DE CAJÓN)**
> La aplicación de un enfoque **sistemático, disciplinado y cuantificable** al **desarrollo, la operación y el mantenimiento** del software.
> SWEBOK p. 38

Sistemático: él no tenía método. Disciplinado: no seguía reglas. Cuantificable: nunca midió nada. Y la definición no termina en «desarrollo»: incluye **operación y mantenimiento**, que fue justo lo que reventó.

Las verdades de Robertson

Antes de empezar, lees el capítulo 1 de Robertson, «Some Fundamental Truths» (p. 32–43). Son 11 verdades. Las que más vas a usar con Doña Rosa:

- **Los requisitos no tratan de escribir documentos**: tratan de descubrir el problema real del negocio (verdad 1).
- El software debe ser **valioso para su dueño**, el que paga: el beneficio tiene que superar el costo (verdad 2).
- **Construir software no es lo mismo que resolver un problema de negocio** (verdad 4). El sobrino construyó software y el problema siguió ahí.

- Los requisitos no tienen que escribirse, pero **sí tienen que llegar a quienes construyen** (verdad 5).
- **El cliente no siempre te da la respuesta correcta** (verdad 6). Robertson dice que trates a cada stakeholder como si pudiera ser Pinocho.
- Puedes ser tan iterativo como quieras, pero **igual tienes que entender lo que el negocio necesita** (verdad 8). Tampoco hay bala de plata (verdad 9).
- Los requisitos **deben ser medibles y comprobables**. Si no puedes medirlo, «no es un requisito, es una idea ociosa» (verdad 10).

El SWEBOK agrega: la ciencia descubre; la ingeniería aplica ese conocimiento a problemas reales **con recursos limitados** (p. 39). Doña Rosa no tiene presupuesto infinito, y por eso toda ingeniería incluye economía.

CAPÍTULO 1 · SWEBOK KA 1 · LEFFINGWELL CAP. 7

La primera reunión

Qué es un requisito, los dos problemas que arruinan proyectos y quiénes son los stakeholders.

Le preguntas a Doña Rosa qué quiere.

> —Una app para que me pidan pan. Que sea rápida. Y que no cobre doble, mijo, por favor.

> **SWEBOK · REQUISITO DE SOFTWARE**
> Una propiedad que el software debe tener para resolver un problema del mundo real. Formalmente: una condición o capacidad que un usuario necesita para lograr un objetivo, o que el sistema debe cumplir para satisfacer un contrato, estándar u otro documento formal.
> SWEBOK p. 44–45, sección 1.1

Robertson lo dice más corto: un requisito es algo que el producto debe **hacer** para apoyar el negocio del dueño, o una **cualidad** que debe tener para ser aceptable (cap. 1, p. 40).

En lo que dijo Doña Rosa ya están **los dos grandes problemas** de los requisitos según el SWEBOK (p. 43):

- **Ambigüedad**: «que sea rápida». Admite muchas interpretaciones y solo una es la que ella tiene en la cabeza.
- **Incompletitud**: lo que *no* dijo. No mencionó que la mitad de sus clientes paga en efectivo, porque para ella es obvio.

Cada requisito genera decisiones de diseño, código y pruebas. Si uno está mal y lo descubres tarde, el retrabajo crece en cascada (p. 43).

No solo Doña Rosa opina

También importan Don Lucho, el cajero; los estudiantes que piden; el repartidor; y la agencia de control sanitario.

> **SWEBOK · STAKEHOLDER**
> Cualquier persona, grupo u organización que participa en el proyecto, que es **afectada** por su resultado o que puede **influir** en él.
> SWEBOK p. 48–49, sección 2.1

El SWEBOK incluye incluso a **los que pueden salir perjudicados si el proyecto tiene éxito** (p. 49). Si la app funciona muy bien, quizá Doña Rosa ya no necesite a Don

Lucho en caja. Don Lucho lo sabe, y por eso no está muy entusiasmado con tus entrevistas.

Leffingwell (cap. 7, p. 156–157) ordena el asunto en dos clases:

> **LEFFINGWELL CAP. 7 · STAKEHOLDERS DE SISTEMA Y DE PROYECTO**
> **Stakeholder del sistema**: quien usa el sistema directamente, quien trabaja con los resultados de quienes lo usan, o quien se ve afectado por su despliegue y operación. **Stakeholder del proyecto**: quien tiene interés en el presupuesto y el cronograma, en cómo se desarrolla, o quien va a vender, instalar o mantener el sistema.
> Leffingwell, *Agile Software Requirements*, cap. 7, p. 157 (pág. impresa 120)

En la panadería: Don Lucho y los estudiantes son del sistema; la hija de Doña Rosa, que pone la plata, es del proyecto. Leffingwell además clasifica a los del sistema por grados (p. 162): **primer grado**, los que usan el sistema (estudiantes, Don Lucho); **segundo grado**, los que usan sus resultados (la hija, que compra harina según los pedidos); **tercer grado**, los que lo instalan o le dan soporte (tú, cuando se cae).

Y no todos participan igual (p. 158): a unos **solo se les informa**, a otros **se les consulta**, otros son **socios del desarrollo** y otros **controlan el resultado** (Doña Rosa). El que junta todas esas voces en un solo backlog priorizado es el **Product Owner**.

Personas

Para no diseñar para un «usuario» genérico, Leffingwell usa **personas** (p. 163–165): perfiles concretos de usuarios. La **persona primaria** necesita una interfaz diseñada específicamente para ella; la **secundaria** puede usar la interfaz de la primaria. Aquí, la primaria es Valeria, estudiante de 19 años que pide desde el celular entre clases. Don Lucho, que solo revisa pedidos, es secundaria.

Reglas de Leffingwell: no inventes personas de la nada; descúbrelas en la investigación. Y si te salen **más de tres personas primarias**, el alcance es demasiado grande: divide el sistema (p. 165).

CAPÍTULO 2 · SWEBOK KA 1 · ELICITACIÓN

Sacarle la verdad a la gente

Qué es elicitar, las técnicas y los 5 porqués.

> **SWEBOK · ELICITACIÓN**
> Actividad cuyo objetivo es hacer aflorar requisitos candidatos (también se llama captura o descubrimiento). Reduce la incompletitud y **no es pasiva**: el ingeniero tiene que esforzarse, porque los usuarios no saben describir su trabajo o se callan cosas.
> SWEBOK p. 48–49, sección 2

Pruebas varias técnicas (SWEBOK p. 49):

- **Entrevistas** a Doña Rosa y a Don Lucho, por separado.
- **Cuestionarios** a 100 estudiantes: ¿a qué hora piden?, ¿cómo pagan?
- **Observación**: te paras en la panadería a las 7 a. m. y descubres que Don Lucho anota los pedidos especiales en un cuaderno, porque «el sistema de antes se los comía».
- **Apprenticing** (aprender haciendo): trabajas un turno. A las 4 a. m. amasando entiendes por qué los pedidos grandes deben llegar el día anterior.
- **Prototipos**: les muestras un dibujo de la app y miras dónde se confunden.
- **Talleres de escritura de historias** y **story mapping** (los verás en el capítulo 6).

Rubin (cap. 5, p. 140) tiene una frase que vale para todo este capítulo: **los usuarios son mejores críticos que autores**. Si les preguntas «¿qué quieres?», no saben. Si les muestras algo, te dicen enseguida qué está mal.

Los 5 porqués

Doña Rosa pide un botón para exportar los pedidos a Excel.

—¿Para qué el Excel? —Para mandárselo a mi hija.

—¿Y para qué se lo manda? —Porque ella compra la harina.

—¿Y por qué necesita el Excel? —Porque no tiene acceso a la app.

El problema real era que la hija necesitaba ver los pedidos. El Excel era una **solución** disfrazada de requisito. Preguntar «¿por qué?» hasta llegar al problema de verdad son los **5 porqués**; casi siempre bastan 2 o 3 vueltas (SWEBOK p. 50). Robertson lo llamaría «encontrar el problema real» (cap. 1).

CAPÍTULO 3 · SWEBOK KA 1 · ROBERTSON CAPS. 10 Y 11

Ordenar el cuaderno

Funcionales, no funcionales y restricciones, según el SWEBOK y según Robertson.

Tienes 60 frases mezcladas. Primero separas las del **producto** («calcular el total del pedido») de las del **proyecto** («entregar en diciembre con dos personas y $800»), como pide el SWEBOK (p. 45). Luego divides las del producto:

> **SWEBOK · REQUISITO FUNCIONAL**
> Especifica **comportamientos observables** del software: las **políticas** que debe hacer cumplir y los **procesos** que debe realizar.
> SWEBOK p. 46, sección 1.4

Política: «un pedido no puede cobrarse dos veces». Proceso: «registrar un pedido y calcular su total».

> **SWEBOK · REQUISITO NO FUNCIONAL**
> Restringe las **tecnologías** de la implementación. Se divide en **restricciones de tecnología** (nombra o prohíbe una tecnología: «debe funcionar en Android») y **calidad de servicio** (un nivel de desempeño: «confirmar el pedido en menos de 3 segundos»).
> SWEBOK p. 46, secciones 1.5 a 1.7

El truco del SWEBOK: el **filtro de la tecnología perfecta** (p. 47). Imagina una computadora mágica, infinitamente rápida, gratis y que nunca falla. Lo que todavía tendrías que pedir («registrar el pedido») es funcional. Lo que deja de importar («responder en 3 segundos») es no funcional.

La versión de Robertson

> **ROBERTSON CAPS. 10 Y 11 · FUNCIONALES VS. NO FUNCIONALES**
> Un requisito **funcional** especifica lo que el producto debe **hacer**: las acciones que realiza para cumplir su razón de existir, expresadas, en lo posible, sin tecnología. Un requisito **no funcional** describe las **cualidades** que el producto debe tener: **qué tan bien** hace lo que hace (rápido, usable, seguro, atractivo).
> Robertson, *Mastering the Requirements Process*, cap. 10 p. 254 (223) y cap. 11 p. 276–279 (245–248)

La forma más fácil de recordarlo, según Robertson: **los funcionales son verbos y los no funcionales son adjetivos** (p. 279). «Registrar el pedido» es un verbo. «En medio segundo» o «solo el supervisor puede modificarlo» son adjetivos del trabajo.

Ojo con un detalle fino: un no funcional **no cambia la funcionalidad esencial**, pero puede **agregar funcionalidad**. Si solo Doña Rosa puede cambiar precios (seguridad), la app necesita un login. Esa función existe por una razón no funcional (p. 278–279). El SWEBOK dice lo mismo: un RNF puede inducir RF (p. 47).

Robertson tiene además un tercer tipo: las **restricciones** (cap. 1, p. 42), que son requisitos globales. Pueden limitar el proyecto («debe estar lista antes de diciembre») o el diseño («debe ser app de Android y iPhone»).

Y trae una historia de terror (p. 276): un sistema de help desk que hacía **todo** lo funcional bien, y los usuarios se negaron a usarlo. Tenía otra apariencia que sus otras nueve aplicaciones, terminología rara, una base de datos que no aguantaba el volumen y hasta íconos ofensivos. Nadie había preguntado por los no funcionales. Lo llaman producto «estilo soviético».

Los tipos de no funcionales de Robertson (plantilla Volere): **apariencia** (look and feel), **usabilidad**, **rendimiento**, **operacionales y de ambiente**, **mantenibilidad y soporte**, **seguridad**, **culturales** y **legales**. Y cada uno necesita un **fit criterion**: una medida que diga si se cumple (cap. 11; plantilla en p. 466–487).

Tres clasificaciones, no te confundas: SWEBOK = tecnología / calidad de servicio. Sommerville (clase) = producto / organizacional / externo. Robertson = funcionales / no funcionales (8 tipos Volere) / restricciones. Responde con la del autor que te pregunten.

CAPÍTULO 4 · SWEBOK KA 1 · WIEGERS CAP. 11

¿Esto está bien escrito?

Qué hace excelente a un requisito y las trampas del lenguaje.

Agarras una frase: «La app debe ser rápida, bonita y avisar al cliente y/o a Doña Rosa». La miras con los ojos de Wiegers.

> **WIEGERS CAP. 11 · CARACTERÍSTICAS DE UN REQUISITO EXCELENTE**
> Cada requisito debe ser: **completo**, **correcto**, **factible**, **necesario**, **priorizado**, **no ambiguo** y **verificable**. El conjunto debe ser: **completo**, **consistente**, **modificable** y **trazable**.
> Wiegers y Beatty, *Software Requirements*, cap. 11, p. 236–240 (203–207)

Aplicado a tu frase:

- **No ambiguo**: «rápida» falla. Wiegers distingue dos ambigüedades: la que tú mismo ves y la peor, cuando cada lector entiende algo distinto y todos creen que está claro (p. 238).
- **Verificable**: ¿cómo pruebas «bonita»? Si no se puede probar, cumplirlo es cuestión de opinión.
- **Completo**: si falta información, se marca **TBD** (por definir) y se resuelve antes de construir (p. 237).
- **Necesario**: cada requisito debe tener una buena respuesta a «¿por qué está aquí?».

El SWEBOK da una lista parecida (p. 50): no ambiguo, verificable, vinculante, **atómico**, que use el vocabulario del stakeholder; y el conjunto completo, conciso, consistente y factible.

Cómo se escribe (la plantilla)

Wiegers propone escribir en **voz activa**, con una palabra clave fija (**«deberá»**, en inglés *shall*) y una de estas dos formas (p. 240–241):

- Desde el sistema (plantilla EARS): **[precondición] [evento disparador] el sistema deberá [respuesta]**. «Si el pedido fue pagado, cuando el cliente toque “pagar”, el sistema deberá mostrar “pedido ya pagado”».
- Desde el usuario: **El [clase de usuario] deberá poder [hacer algo] [a un objeto] [condiciones]**. «El Cajero deberá poder marcar un pedido como entregado».

Wiegers recomienda **no mezclar** «deberá», «debería» y «podría» como si significaran prioridades distintas; mejor siempre «deberá» y la prioridad aparte. Un consultor sugería leer cada «debería» como «probablemente no» (p. 242). Si el resultado no es aceptable, cambia la palabra.

Las trampas del lenguaje (p. 243–250)

- **Palabras difusas**: rápido, amigable, fácil, eficiente, robusto, «etc.», «entre otros», «normalmente». Wiegers tiene una tabla entera de ellas (tabla 11-2, p. 247).
- **«y/o»**: deja la interpretación al lector. Di «y», «o» o «cualquier combinación».
- **«salvo», «excepto», «pero»**: esconden dos requisitos. «Se cobra con la tarjeta, salvo que esté vencida». ¿Y qué pasa si está vencida? Sepáralo en dos.
- **Varios requisitos en uno**: si se necesitan pruebas distintas, son requisitos distintos.
- **Voz pasiva**: «el pedido será actualizado». ¿Por quién? ¿El sistema o Don Lucho?
- **Valores límite**: «hasta 5 panes sin anticipación; de 5 a 10, con un día». ¿Y si son exactamente 5? Usa «5 o menos» y «más de 5 hasta 10».
- **Requisitos negativos**: «impedir pedidos si no hay stock». Mejor en positivo: «permitir pedidos solo si hay stock».
- **Simetría y excepciones olvidadas**: si se puede guardar un pedido a medias, ¿se puede recuperar? Si falla el pago, ¿qué pasa?

Cuando dos stakeholders chocan

Doña Rosa quiere que paguen antes de recoger; Don Lucho, al recoger. El SWEBOK propone **negociar** (sin decidir tú solo) o hacer el sistema **configurable**, separando lo que todos aceptan de lo que varía (p. 52). Aquí: el cliente elige «pagar ahora» o «pagar al recoger».

CAPÍTULO 5 · WIEGERS CAP. 10 · SWEBOK KA 1

El documento

La SRS, cómo etiquetar requisitos, las formas de especificar y la validación.

La hija de Doña Rosa quiere «algo por escrito». Wiegers abre su capítulo 10 con una advertencia: un gerente juntó las ideas de 60 personas en 100 páginas y lo llamó especificación. No lo era: **era una pila de información** (p. 214).

> **WIEGERS CAP. 10 · SRS (ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE)**
> Documento que establece las **funciones y capacidades** que el sistema debe proveer, sus **características** y las **restricciones** que debe respetar. Es la base para planificar, diseñar, programar, probar y documentar. **No** debe contener detalles de diseño, construcción, pruebas o gestión, salvo restricciones conocidas.
> Wiegers y Beatty, cap. 10, p. 216 (183)

Lo leen muchos: el cliente, el gerente que estima, los desarrolladores, los testers, soporte, quien escribe el manual, los abogados (p. 217). Por eso Wiegers dice: si una capacidad no está en el acuerdo de requisitos, **nadie debería esperar que aparezca en el producto**.

Cuando la SRS se revisa y se aprueba, queda como **línea base** (baselining, p. 218): desde ahí se trabaja sobre un acuerdo firmado.

Etiquetar cada requisito

Cada requisito necesita un **identificador único y persistente** (p. 219–221). Tres formas:

- **Número de secuencia**: RF-26. Simple, pero no dice nada del contenido.
- **Numeración jerárquica**: 3.2.4.3. Familiar, pero los números cambian si insertas algo.
- **Etiquetas textuales jerárquicas** (de Gilb): **Pedido.ConfirmarPago**. Dicen qué es y no cambian.

La plantilla de Wiegers (figura 10-2, p. 224)

1. Introducción · 2. Descripción general · 3. Funcionalidades del sistema · 4. Requisitos de datos · 5. Interfaces externas · 6. Atributos de calidad · 7. Internacionalización · 8. Otros requisitos. Más glosario y modelos de análisis. Es pariente de la IEEE 830 y la ISO 29148 que usa tu proyecto.

¿Y en ágil? Wiegers dice que los equipos ágiles igual necesitan esta información, solo que no la juntan en un documento único: la reparten en historias y pruebas de aceptación (p. 232).

Otras formas de especificar (SWEBOK p. 53–56)

De más suelta a más precisa: **lenguaje natural libre** («el sistema deberá…», reglas de negocio); **lenguaje estructurado** (actor-acción, casos de uso, historias de usuario); **criterios de aceptación** (BDD: «Dado… cuando… entonces…», que ataca la ambigüedad porque una prueba no admite interpretaciones); y **modelos** (UML, diagramas de estados).

¿Es lo que Doña Rosa necesitaba?

> **SWEBOK · VALIDACIÓN DE REQUISITOS**
> Ganar confianza en que los requisitos representan las **verdaderas necesidades** de los stakeholders. Tres métodos: **revisiones**, **simulación o ejecución** y **prototipos**.
> SWEBOK p. 57–58, sección 5

Con el prototipo pasa lo que advierte el SWEBOK (p. 58): Doña Rosa pasa 40 minutos discutiendo el color del botón y cero minutos revisando si el total está bien calculado.

Robertson, por su parte, revisa en dos niveles: la **Quality Gateway** prueba cada requisito individual (bien escrito, no ambiguo, dentro del alcance, comprobable, trazable y sin «enchapado en oro»), y la **revisión de la especificación** busca lo que falta, prioriza y detecta conflictos (cap. 17, p. 402–403).

CAPÍTULO 6 · RUBIN CAP. 5 · LEFFINGWELL CAP. 6 · COHN

Las tarjetas en la pared

Historias de usuario: las 3 C, los niveles, INVEST, cómo dividirlas y los spikes.

El equipo decide trabajar con Scrum. Entonces la SRS gigante se reemplaza por tarjetas. Rubin explica el cambio de mentalidad (cap. 5, p. 124): en el desarrollo secuencial los requisitos son **no negociables y detallados desde el inicio**; en Scrum se detallan **justo a tiempo y justo lo necesario**, conversando. Para eso se usan los **elementos del product backlog** (PBI), y la forma más común de escribirlos son las historias de usuario.

> **LEFFINGWELL CAP. 6 · HISTORIA DE USUARIO**
> Una **declaración breve de intención** que describe algo que el sistema necesita hacer para el usuario. No es una especificación detallada («el sistema deberá…»), sino una expresión **negociable** de intención.
> Leffingwell, cap. 6, p. 137–138 (100–101)

La plantilla, que Cohn popularizó en *User Stories Applied* (Leffingwell p. 140, Rubin p. 128):

Como <rol>, quiero <objetivo> para <beneficio>

«Como estudiante con clase a las 7, quiero pedir mi pan la noche anterior para no llegar tarde». Leffingwell explica por qué funciona: el rol da la perspectiva del usuario, la actividad es lo que hace el sistema y el beneficio dice **por qué**, lo que a veces revela una forma más barata de lograr lo mismo (p. 141).

Las 3 C

> **RUBIN CAP. 5 · LEFFINGWELL CAP. 6 · COHN CAP. 1 · LAS 3 C (RON JEFFRIES)**
> **Card** (tarjeta): 2 o 3 frases que capturan la intención; es un recordatorio, no el requisito completo. **Conversation** (conversación): el diálogo continuo entre equipo, Product Owner y stakeholders donde salen los detalles; la tarjeta es «una promesa de conversar». **Confirmation** (confirmación): las **condiciones de satisfacción**, pruebas de aceptación que confirman que la historia está bien hecha.
> Rubin p. 128–130 (83–85) · Leffingwell p. 139–140 (102–103)

En la tarjeta del pedido nocturno: al reverso, la confirmación dice «verificar que se puede pedir hasta las 22:00», «verificar que el pedido aparece a Don Lucho a las 5:00», «verificar que no se aceptan pedidos de más de 50 panes».

Historias de todos los tamaños

Rubin muestra una jerarquía (p. 131–133): **épica** (meses; demasiado grande para un sprint), **feature** (semanas), **historia** o historia «sprintable» (días) y, debajo, **tareas** (horas; el *cómo*, no el *qué*). Un **tema** es un grupo de historias relacionadas. En la panadería: «Comprar en la panadería» es una épica; «Pagar» es un tema; «Pagar con transferencia» es una historia; «crear la tabla de pagos» es una tarea.

INVEST: ¿la historia es buena?

> **RUBIN CAP. 5 · LEFFINGWELL CAP. 6 · COHN CAP. 2 · INVEST (BILL WAKE)**
> **I**ndependiente (pocas dependencias con otras) · **N**egociable (dice el qué y el porqué, no el cómo) · **V**aliosa (para el cliente o el usuario; la más importante) · **E**stimable (el equipo puede calcular su tamaño) · **S**mall, pequeña (cabe en un sprint) · **T**estable (pasa o no pasa sus pruebas).
> Rubin p. 133–137 (88–92) · Leffingwell p. 142–148 (105–111)

Detalles que suelen preguntar:

- **Valiosa**: «Como desarrollador, quiero migrar la base de datos» es una historia técnica. Solo entra al backlog si el Product Owner entiende su valor; si no, es una **tarea** (Rubin p. 135–136). Leffingwell propone reescribirla desde el valor: «Como cliente, quiero ver siempre el mismo mensaje de error claro…» (p. 145).
- **Valiosa también significa vertical**: Leffingwell usa la metáfora del pastel de capas (base de datos, lógica, pantalla). Una historia debe cortar el pastel **de arriba abajo**; una capa sola no le sirve a nadie (p. 144).
- **Pequeña** depende de cuándo la vas a hacer: una épica que vas a trabajar en un año está bien que sea grande (Rubin p. 137).
- **Testable**: palabras como «rápido», «gestionar» o «bonito» son imposibles de probar (Leffingwell p. 148).

Cuando la historia es muy grande

«Como cliente, quiero gestionar mis pedidos». «Gestionar» es la palabra trampa: son muchas operaciones. Leffingwell da **10 patrones para dividir** (tabla 6-1, p. 149–150): por pasos del flujo, variaciones de reglas de negocio, esfuerzo mayor, simple/complejo, variaciones de datos, métodos de entrada, posponer cualidades (primero que funcione, luego que sea rápido), **operaciones CRUD** (crear, ver, editar, cancelar pedido), escenarios de casos de uso y separar un **spike**.

Cuando nadie sabe cómo hacerlo: spikes

> **LEFFINGWELL CAP. 6 · RUBIN CAP. 5 · SPIKE / HISTORIA DE ADQUISICIÓN DE CONOCIMIENTO**
>
> Historia especial para **reducir riesgo e incertidumbre**: investigar una tecnología, entender una historia demasiado grande o probar un enfoque. Produce **información** (una decisión, un prototipo), no código funcionando. Puede ser **técnico** (cómo construirlo) o **funcional** (cómo interactuará el usuario). Debe ser la excepción, no la regla.
>
> Leffingwell p. 151–153 (114–116) · Rubin p. 138–140 (93–95)

Rubin lo justifica con plata (p. 139–140): si probar dos arquitecturas cuesta $10K y equivocarse costaría $500K, vale la pena (la información vale $250K). Si equivocarse costara solo $15K, mejor tira una moneda.

No funcionales en ágil

Rubin dice que los no funcionales son **restricciones de todo el sistema** y que el mejor lugar para ellos es la **Definition of Done**: si «debe funcionar en Chrome y Safari» está ahí, ninguna historia está terminada hasta probarse en ambos (p. 138).

¿De dónde salen las historias?

Rubin propone dos técnicas (p. 140–143). El **taller de escritura de historias**: Product Owner, equipo y stakeholders hacen lluvia de ideas por unas horas, empezando por los roles de usuario. Y el **story mapping** (Jeff Patton): las épicas arriba, los temas ordenados de izquierda a derecha según el flujo del usuario y, debajo de cada uno, las historias ordenadas por prioridad. Es un backlog en dos dimensiones.

Lo que pide Cohn

El libro de Cohn (caps. 1, 2, 7, 12, 13 y 14) **no está en el Aula Virtual ni en el vault**, así que aquí solo va lo que Rubin y Leffingwell citan de él, más el tema de cada capítulo para que sepas qué buscar si consigues el libro. Cap. 1: qué es una historia (las 3 C). Cap. 2: escribir historias (INVEST). Cap. 7: pautas para buenas historias. Cap. 12: qué **no** son las historias (no son requisitos IEEE 830, ni casos de uso, ni escenarios). Cap. 13: por qué usar historias. Cap. 14: señales de historias problemáticas. Si el control pregunta por estos capítulos, responde con las ideas de este capítulo y nombra a Cohn como el autor de la plantilla «Como… quiero… para…».

CAPÍTULO 7 · RUBIN CAPS. 2 Y 4 · SLIDES 03B

El primer sprint

Scrum completo: roles, actividades, artefactos, sprints y la Definition of Done.

> **RUBIN CAP. 2 · QUÉ ES SCRUM**
> Scrum **no es un proceso estandarizado** de pasos que garantizan un buen producto a tiempo. Es un **marco para organizar y gestionar el trabajo**, basado en valores, principios y prácticas, sobre el que cada organización agrega sus propias prácticas de ingeniería. El resultado es «una versión de Scrum que es únicamente tuya».
> Rubin, *Essential Scrum*, cap. 2, p. 58 (13) · slide 03b p. 5

Rubin lo compara con un edificio: los valores, principios y prácticas son los cimientos y las paredes. No los puedes quitar sin que se caiga, pero puedes decorar por dentro (p. 58). Las slides añaden que Scrum trabaja en iteraciones cortas con duración fija, de una semana a un mes (p. 4).

Los tres roles

- **Product Owner**: el punto central de liderazgo del producto. Es la **única autoridad** que decide qué se construye y en qué orden, y es responsable del éxito de la solución (p. 60). En la panadería no es Doña Rosa: es tu compañera Andrea, que habla con ella todos los días.
- **ScrumMaster**: un coach. Ayuda a entender y vivir los valores y prácticas de Scrum, protege al equipo de interferencias y elimina impedimentos. **No tiene autoridad para controlar al equipo**; no es un jefe de proyecto. Es líder, no gerente (p. 61).
- **Equipo de desarrollo**: grupo **multifuncional** (programadores, testers, diseñadores) que se **autoorganiza** para lograr el objetivo. Típicamente **de 5 a 9 personas**; si son 35, se arman 4 o más equipos (p. 61).

Resumen de Rubin (p. 60): el Product Owner decide **qué** y en qué orden; el ScrumMaster guía el **proceso**; el equipo decide **cómo**.

El recorrido de un sprint (figura 2.3, p. 62)

1. Andrea tiene la visión de la app. Con el **grooming** (refinamiento) la parte en features: crear, refinar, estimar y priorizar elementos. Esa lista ordenada es el **product backlog** (p. 63–64). Puede tener features, cambios, defectos y mejoras técnicas, y cambia todo el tiempo.
2. Antes de ordenar hay que saber el tamaño: se usan medidas relativas como **story points** o días ideales. Si «pagar» mide 2 y «delivery» mide 8, delivery es unas 4 veces

más grande (p. 65).

3. En el **sprint planning**, Product Owner, equipo y ScrumMaster acuerdan un **sprint goal** («los estudiantes pueden pedir y pagar desde el celular») y eligen lo que el equipo puede terminar a un **ritmo sostenible** (p. 66–67).
4. El equipo divide cada feature en tareas estimadas en horas. Esas tareas, más los elementos elegidos, forman el **sprint backlog**. Según las slides, **ninguna tarea debe pasar de 16 horas** (p. 25). Para un sprint de 2 a 4 semanas, la planificación toma de 4 a 8 horas (Rubin p. 67).
5. **Ejecución del sprint**: nadie le dice al equipo en qué orden hacer las tareas; se autoorganiza (p. 68).
6. Cada día, el **daily scrum**: 15 minutos o menos, idealmente a la misma hora. Cada uno responde: ¿qué hice desde el último daily?, ¿qué haré hasta el próximo?, ¿qué me bloquea? **No es para resolver problemas ni es una reunión de estado para un jefe**: es inspección, sincronización y planificación diaria (p. 69).
7. Al final queda un **incremento de producto potencialmente entregable**: terminado según la Definition of Done. «Potencialmente» significa que **se podría** entregar; entregarlo de verdad es una decisión de negocio (p. 70–71).
8. **Sprint review**: equipo y stakeholders inspeccionan **el producto**. Doña Rosa prueba la app y opina (p. 71).
9. **Sprint retrospective**: el equipo inspecciona **el proceso**: qué funcionó con Scrum y qué no, y se compromete a mejoras para el próximo sprint (p. 72–73).

Para el control: **3 roles** (PO, ScrumMaster, equipo); **actividades** (sprint, planning, grooming, ejecución, daily, review, retro); **3 artefactos** (product backlog, sprint backlog, incremento potencialmente entregable) (figura 2.1, p. 59). La review mira el producto y la retro mira el proceso.

Los **8 valores** de Scrum según Rubin y las slides (p. 58; slide p. 40): honestidad, apertura, valentía, respeto, enfoque, confianza, empoderamiento y colaboración.

Dato curioso de Rubin (p. 70): Scrum usaba «cerdos» y «gallinas». En un desayuno de huevos con jamón, la gallina está involucrada, pero el cerdo está comprometido. En el daily solo hablan los cerdos. (Ese día Don Lucho fue gallina.)

Las reglas del sprint (Rubin cap. 4)

> **RUBIN CAP. 4 · SPRINT**
> Los sprints son el **esqueleto** de Scrum. Son **timeboxed** (fecha de inicio y fin fijas), **cortos** (de una semana a un mes), de **duración constante**, sin **cambios que alteren el objetivo** una vez iniciados, y deben terminar según la **Definition of Done** acordada.
> Rubin cap. 4, p. 106–107 (61–62)

¿Por qué timebox? (p. 107–109): limita el trabajo en proceso (WIP), obliga a priorizar, demuestra avance, evita el perfeccionismo innecesario, motiva a terminar y mejora la predictibilidad.

¿Por qué cortos? (p. 109–111): es más fácil planificar, la retroalimentación llega rápido, el retorno de inversión llega antes, el error queda acotado («si nos equivocamos, perdimos solo dos semanas»), el entusiasmo se mantiene y hay más puntos de control. Rubin cuenta que en IBM llamaban «hervir el océano» a los proyectos eternos.

¿Por qué duración constante? (p. 112–113): da **cadencia**, un ritmo predecible, y permite calcular la **velocidad** (cuánto termina el equipo por sprint). No terminar a tiempo **no es razón** para alargar el sprint.

Doña Rosa quiere cambiar el sprint

A mitad del sprint, Doña Rosa pide agregar un programa de puntos para clientes frecuentes. Rubin distingue (p. 114–115):

- **Cambio**: altera trabajo o recursos y puede generar desperdicio o aumentar el alcance. El programa de puntos es un cambio, y va al product backlog para un sprint futuro. Rubin usa el ejemplo de alguien que pide buscar sospechosos por foto de sus tatuajes a mitad de sprint.
- **Aclaración**: un detalle que ayuda a lograr el objetivo. «¿Ordeno los pedidos por hora o por nombre?» «Por hora». Eso sí se permite.

Es una regla, no una ley: si pasa algo económicamente grave (la competencia lanza algo que invalida el sprint, o se cae producción), se puede cambiar. Y si el objetivo queda completamente inválido, el sprint se **termina anormalmente**: se hace retro y se planifica uno nuevo. Es el último recurso (p. 117–118).

¿Cuándo algo está terminado?

> **RUBIN CAP. 4 · DEFINITION OF DONE VS. CRITERIOS DE ACEPTACIÓN**
> La **Definition of Done** es un checklist del trabajo que el equipo debe completar para declarar su trabajo potencialmente entregable (diseño revisado, código revisado, probado, integrado, documentado…). Aplica a **todos** los elementos. Los **criterios de aceptación** (condiciones de satisfacción) son **propios de cada elemento**, los define el Product Owner. Un elemento está terminado solo si cumple **ambos**.
>
> Rubin cap. 4, p. 119–122 (74–77)

Ejemplo: «pagar con tarjeta» tiene como criterio de aceptación «funciona con Visa, MasterCard y Amex». Pero además debe cumplir la DoD: probado, integrado, en el servidor. Y si el último día queda un defecto importante, **no está terminado**: vuelve al backlog; el sprint no se alarga (p. 120). La DoD puede ir creciendo con el tiempo (p. 121). Rubin cierra con su hijo: para él, «terminé la tarea» significaba «hice lo que estaba dispuesto a hacer» (p. 123).

Burndown, beneficios y límites

El **burndown** es un gráfico del trabajo pendiente que va bajando sprint a sprint (slides p. 29–31). Beneficios de Scrum según las slides (p. 50): un producto potencialmente entregable en cada sprint, requisitos inestables detectados pronto, todo el equipo conoce el estado del proyecto y se construye confianza con el cliente. Limitaciones (p. 51): más difícil en proyectos grandes, requiere un equipo con habilidades transversales, supone un cliente muy activo y no genera parte de la documentación que exigen algunos estándares.

Ojo: el SWEBOK (p. 242) dice que el Scrum Master «gestiona las actividades». Rubin y las slides dicen que es un coach sin autoridad sobre el equipo. Si preguntan por Scrum según Rubin o la clase, usa la versión de Rubin.

CAPÍTULO 8 · SWEBOK KA 1 · GESTIÓN · ROBERTSON CAP. 14

Doña Rosa cambia de idea

Mantener el acuerdo: depurar, controlar cambios, priorizar y rastrear.

El SWEBOK lo resume así (p. 48): **desarrollar requisitos es llegar a un acuerdo** sobre qué construir; **gestionarlos es mantener ese acuerdo en el tiempo**.

> —Mijo, ¿y si la app también tiene un juego para los niños, una inteligencia artificial que recomiende panes y modo oscuro?

- **Depuración (scrubbing)**: buscar el conjunto más pequeño de requisitos que resuelva el problema. El juego y la IA se van (SWEBOK p. 58).
- **Control de cambios**: se pide el cambio, se analiza el impacto, alguien decide y se avisa. En ágil, el cambio entra al product backlog y espera su turno (p. 59).
- **Ajuste de alcance**: que lo prometido quepa en tiempo, plata y personas (p. 59).
- **Kano**: no basta con ver cuánto le gusta algo al usuario, hay que ver cuánto le enoja que falte. El modo oscuro gusta, pero no poder pagar en efectivo enfurece (p. 60).
- **Trazabilidad**: seguir el hilo botón → diseño → requisito → entrevista, y al revés para medir el impacto de un cambio (p. 60–61).

Robertson (cap. 14, p. 354–355) agrega que lo iterativo no elimina los requisitos: el negocio genera necesidades todo el tiempo, se analizan continuamente y se descubren requisitos **en paralelo** con el desarrollo, entregando en pequeños incrementos con mucha conversación con el cliente.

CAPÍTULO 9 · SWEBOK KA 10 Y 11

¿Por qué Scrum y no cascada?

Proceso, ciclo de vida, modelos y ágil según el SWEBOK.

> **SWEBOK · PROCESO Y CICLO DE VIDA**
> Un **proceso** es un conjunto de actividades relacionadas que **transforman entradas en salidas** (actividad = conjunto de tareas; tarea = acción concreta). Un **ciclo de vida** es la evolución de un sistema **desde que se concibe hasta que se retira**.
> SWEBOK p. 223

- **Cascada**: predictiva y dirigida por documentos; una fase no empieza hasta que termina la anterior (p. 226). Con Doña Rosa, que cambia de idea cada martes, sería un suicidio lento.
- **Espiral** (Boehm): evolutiva y **dirigida por riesgos**; cada vuelta ataca primero lo que más miedo da (p. 226).
- **Incremental**: cada entrega agrega funcionalidad (p. 225). **Prototipado rápido**: prototipos temprano para recibir opiniones. **Proceso Unificado/RUP**: iterativo e incremental (p. 226).
- **Ágil**: abierto al cambio, comunicación con el cliente, entregas pequeñas. Tres mitos que desmiente el SWEBOK (p. 227): ágil **no es un método** (Scrum sí), **no es más rápido porque no documenta** y **no es desordenado**.

> **SWEBOK · SCRUM**
> Método ágil orientado a la gestión de proyectos: sprints de máximo 30 días, Scrum Master, Product Owner con su product backlog, una versión probada y liberada en cada sprint, y Daily Scrum.
> SWEBOK p. 242

Lo mismo que dice Rubin, pero en cinco líneas. Si te preguntan «según el SWEBOK», usa esta tarjeta.

CAPÍTULO 10 · SWEBOK KA 12

El croissant de un centavo

Calidad, y la diferencia entre error, defecto y falla.

Una noche, cansado, escribes el precio del croissant como `0.01` en vez de `1.00`. Dos semanas después alguien lo descubre, lo comparte en el grupo de la carrera, y en una mañana se venden 400 croissants a un centavo. El SWEBOK le pone nombre a cada parte (p. 248):

- **Error**: la acción humana equivocada (tú, escribiendo mal).
- **Defecto**: la imperfección que queda en el software (el `0.01` durmiendo en la base de datos).
- **Falla**: el sistema no hace lo que debe y alguien lo ve (los 400 croissants).

> **SWEBOK · CALIDAD DE SOFTWARE**
> El grado en que un producto de software cumple los requisitos establecidos; pero la calidad depende de qué tan bien esos requisitos representan las **necesidades, deseos y expectativas** de los stakeholders.
> SWEBOK p. 248 (también p. 246)

Una app que cumple todo el documento, pero en el documento nadie puso «pago en efectivo», cumple el papel y no sirve. Calidad es cumplir **lo correcto**.

Verificar es comprobar que construiste **bien** el producto; **validar**, que construiste **el producto correcto** (p. 257). Y el costo de la calidad (p. 249): prevenir y probar cuesta, pero arreglar después cuesta más.

CAPÍTULO 11 · SWEBOK KAS 6, 7, 8, 9 Y 15

La vida después del lanzamiento

Operar, mantener, controlar versiones y decidir con plata.

> **SWEBOK · MANTENIMIENTO**
> La totalidad de actividades para dar **soporte costo-efectivo** al software en operación.
> SWEBOK p. 167; tipos en p. 170–171

Tipos: **correctivo** (arreglar el croissant), **preventivo** (arreglar el 29 de febrero antes de que llegue), **adaptativo** (nueva versión de Android), **aditivo** (agregar delivery), **perfectivo** (mejorar el código de las 3 a. m.) y **de emergencia** (parche un viernes a las 11 p. m.).

> **SWEBOK · GESTIÓN DE CONFIGURACIÓN (SCM)**
> Aplicar gestión de configuración en todo el ciclo de vida para que los elementos del proyecto estén completos y correctos: **identificarlos**, **controlar sus cambios**, **registrar su estado** y **verificar** que cumplen.
> SWEBOK p. 185; línea base en p. 191

Cuando Doña Rosa jura que nunca pidió algo, abres la versión firmada del documento: la **línea base**.

Riesgo (p. 210): la incertidumbre es falta de información; el riesgo es su efecto sobre los objetivos, y puede ser bueno o malo.

> **SWEBOK · ECONOMÍA DE LA INGENIERÍA DE SOFTWARE**
> Alinea las decisiones técnicas con los objetivos del negocio. La economía es la **ciencia de elegir**, no del dinero: ¿invertir aquí da más valor que otra alternativa?
> SWEBOK p. 288

¿Programas tu propio sistema de pagos o usas uno que cobra comisión? Comparas alternativas. Es lo que el profe quiere oír con «¿y contra qué lo evaluaste?».

EPÍLOGO

Ya recorriste todo

Si te piden «8 áreas del SWEBOK», recorre la historia en orden.

La reunión y el cuaderno	**1. Requisitos** (p. 43)
El servidor y la app	**2. Arquitectura** (p. 67) · **3. Diseño** (p. 83)
Programar a las 3 a. m.	**4. Construcción** (p. 99)
Buscar el croissant	**5. Pruebas** (p. 117)
Que la app no se caiga	**6. Operaciones** (p. 151)
Los parches del primer año	**7. Mantenimiento** (p. 167)
«Yo nunca pedí eso»	**8. Gestión de configuración** (p. 185)
Cronograma y riesgos	**9. Gestión** (p. 202)
Elegir Scrum	**10. Proceso** (p. 221) · **11. Modelos y métodos** (p. 233)
Cumplir lo correcto	**12. Calidad** (p. 246)
Que nadie robe tarjetas	**13. Seguridad** (p. 264)
No mentirle a Doña Rosa	**14. Práctica profesional** (p. 274)
«¿Y cuánto cuesta?»	**15. Economía** (p. 288)
Lo que traes de antes	**16–18. Fundamentos** de computación, matemáticos y de ingeniería

¿Qué libro está en qué capítulo de la historia?

Rubin cap. 2 · Scrum Framework	Capítulo 7 (p. 58–73 del PDF)
Rubin cap. 4 · Sprints	Capítulo 7 (p. 106–123)
Rubin cap. 5 · Requirements and User Stories	Capítulo 6 (p. 124–143)
Leffingwell cap. 6 · User Stories	Capítulo 6 (p. 136–155)
Leffingwell cap. 7 · Stakeholders, Personas, UX	Capítulo 1 (p. 156–171)

Wiegers cap. 10 · Documenting the requirements	Capítulo 5 (p. 214–235)
Wiegers cap. 11 · Writing excellent requirements	Capítulo 4 (p. 236–253)
Robertson caps. 1, 10, 11, 14, 16, 17	Prólogo y capítulos 3, 5 y 8
Cohn caps. 1, 2, 7, 12, 13, 14	Capítulo 6 (el libro no está en el Aula Virtual)
Scrum Institute, *The Scrum Framework*	Capítulo 7 (no está en el Aula Virtual; las slides 03b lo citan)

PARA PRACTICAR

Así te van a preguntar

Tapa la respuesta, contesta en voz alta y compara. Fórmula: definición + para qué sirve + un ejemplo.

1. ¿Qué es la ingeniería de software según el SWEBOK?
Aplicar un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software (p. 38). Ej.: hacer la app de una panadería con requisitos validados, pruebas y control de versiones.

2. Diferencie requisito funcional y no funcional (con ejemplo).
Funcional: lo que el producto debe hacer, sus políticas y procesos («registrar el pedido»). No funcional: cualidades, qué tan bien lo hace, o restricciones de tecnología («confirmar en máximo 3 segundos»). Robertson: los funcionales son verbos; los no funcionales, adjetivos.

3. ¿Qué es un stakeholder? Diferencie stakeholder del sistema y del proyecto (Leffingwell).
Quien participa, es afectado o puede influir en el proyecto (SWEBOK p. 48). Del sistema: quien lo usa, usa sus resultados o es afectado por su operación (el cajero). Del proyecto: quien se interesa por presupuesto, cronograma o cómo se desarrolla (el patrocinador) (Leffingwell p. 157).

4. ¿Qué es una persona primaria y una secundaria?
Primaria: usuario que necesita una interfaz diseñada específicamente para él. Secundaria: usa el sistema, pero puede usar la interfaz de la primaria (Leffingwell p. 164). Ej.: el estudiante que pide es primaria; el repartidor que solo consulta, secundaria.

5. ¿Qué son las 3 C de una historia de usuario?
Card: la tarjeta con la intención en pocas frases. Conversation: el diálogo donde salen los detalles. Confirmation: las pruebas de aceptación que confirman que se cumplió (Rubin p. 128–130; Leffingwell p. 139). Ej.: tarjeta «pedir pan la noche anterior», conversación con el PO sobre el horario y confirmación «se acepta hasta las 22:00».

6. ¿Qué significa INVEST? Explique la V.
Independiente, Negociable, Valiosa, Estimable, Small (pequeña), Testable. Valiosa: debe aportar valor al cliente o usuario; una historia puramente técnica solo entra si el PO entiende su valor, y si no, es una tarea (Rubin p. 135).

7. ¿Qué es un spike?
Historia para reducir incertidumbre o riesgo; produce información (una decisión, un prototipo), no código funcionando. Puede ser técnico o funcional (Leffingwell p. 151). Ej.: investigar en un sprint qué pasarela de pagos usar antes de comprometer la historia de pagos.

8. Nombre los roles, las actividades y los artefactos de Scrum.
Roles: Product Owner, ScrumMaster, equipo de desarrollo. Actividades: sprint, sprint planning, grooming, ejecución, daily scrum, review y retrospectiva. Artefactos: product backlog, sprint backlog, incremento potencialmente entregable (Rubin p. 59).

9. Diferencie sprint review y sprint retrospective.
Review: se inspecciona y adapta el **producto**, con stakeholders. Retrospective: se inspecciona y adapta el **proceso**, solo el equipo Scrum (Rubin p. 71–73).

10. ¿Qué es la Definition of Done y en qué se diferencia de los criterios de aceptación?
DoD: checklist de trabajo que se debe completar para que algo sea potencialmente entregable (probado, integrado, documentado); aplica a todos los elementos. Criterios de aceptación: condiciones propias de cada elemento. Un elemento está terminado solo si cumple ambos (Rubin p. 119–122).

11. ¿Se puede cambiar el alcance a mitad del sprint?
No se permiten cambios que alteren el objetivo; sí aclaraciones. Un cambio va al backlog para otro sprint. Es una regla pragmática: si las consecuencias económicas de no cambiar son peores, se cambia, y si el objetivo queda inválido el sprint se termina anormalmente (Rubin p. 114–118).

12. Nombre 4 beneficios del timeboxing.
Limita el trabajo en proceso, obliga a priorizar, demuestra avance, evita el perfeccionismo, motiva a terminar y mejora la predictibilidad (Rubin p. 107–109).

13. ¿Qué es una SRS según Wiegers?
Documento que establece las funciones y capacidades del sistema, sus características y las restricciones que debe respetar; base para planificar, diseñar, programar y probar. No debe incluir diseño ni gestión (cap. 10, p. 216).

14. Nombre las características de un requisito excelente según Wiegers.
Individual: completo, correcto, factible, necesario, priorizado, no ambiguo y verificable. El conjunto: completo, consistente, modificable y trazable (cap. 11, p. 236–240).

15. ¿Qué está mal en «El sistema debe cobrar con la tarjeta, salvo que esté vencida, y avisar rápido al cliente y/o al cajero»?
«Salvo» esconde un segundo requisito (¿qué pasa si está vencida?); «rápido» es ambiguo y no verificable; «y/o» deja la interpretación al lector; y hay varios requisitos en uno (Wiegers cap. 11). Reescritura: «Si la tarjeta está vigente, el sistema deberá cobrar el pedido. Si está vencida, el sistema deberá pedir otra tarjeta. Tras cobrar, el sistema deberá notificar al cliente en máximo 10 segundos».

16. ¿Qué es la calidad de software según el SWEBOK?
El grado en que el software cumple sus requisitos, siempre que esos requisitos representen bien las necesidades de los stakeholders (p. 248).

17. Diferencie error, defecto y falla.
Error: acción humana equivocada. Defecto: imperfección que queda en el software. Falla: el sistema no hace lo que debe y se nota (SWEBOK p. 248). Ej.: escribir mal un precio; el precio erróneo guardado; los clientes comprando a un centavo.

18. ¿Qué es software configuration management?
Aplicar gestión de configuración en todo el ciclo de vida: identificar los elementos, controlar sus cambios, registrar su estado y verificar que cumplen (SWEBOK p. 185). Ej.: Git con versiones etiquetadas y el ERS firmado como línea base.

ÚLTIMO REPASO

La historia en una página

El sobrino hizo código, no software (programas + documentación + datos). IS = sistemática, disciplinada, cuantificable; desarrollo, operación y mantenimiento (SWEBOK p. 38). Robertson: los requisitos tratan de descubrir el problema real; el cliente puede ser Pinocho; si no se puede medir, es una idea ociosa.

La reunión. Requisito = propiedad para resolver un problema real. Problemas: ambigüedad e incompletitud. Stakeholder = participa, es afectado o influye. Leffingwell: del sistema vs. del proyecto; grados 1, 2 y 3; personas primarias y secundarias (más de 3 primarias = alcance muy grande).

El cuaderno. Elicitar = hacer aflorar requisitos; no es pasivo. Usuarios: mejores críticos que autores (Rubin). 5 porqués: el Excel no era el problema.

Ordenar. SWEBOK: funcional (políticas y procesos) vs. no funcional (tecnología / calidad de servicio); filtro de la computadora mágica. Robertson: funcionales = verbos, no funcionales = adjetivos, más restricciones; 8 tipos Volere; fit criterion.

¿Bien escrito? Wiegers: completo, correcto, factible, necesario, priorizado, no ambiguo, verificable; el conjunto completo, consistente, modificable y trazable. «Deberá», voz activa, EARS. Trampas: palabras difusas, y/o, «salvo», límites, negativos, excepciones.

El documento. SRS = funciones, características y restricciones; sin diseño. Etiquetas: secuencia, jerárquica, textual (Pedido.ConfirmarPago). TBD. Línea base. Validar: revisiones, simulación, prototipos.

Las tarjetas. Historia = declaración breve de intención. «Como… quiero… para…». 3 C: card, conversation, confirmation. Épica → feature → historia → tarea; tema = grupo. INVEST. Corte vertical del pastel. 10 patrones para dividir (CRUD). Spike = compra información. No funcionales en la DoD. Taller de historias y story mapping.

El sprint. Scrum = marco, no proceso. PO (qué), ScrumMaster (coach, sin autoridad), equipo de 5 a 9 que se autoorganiza (cómo). Backlog → planning (goal + sprint backlog, tareas ≤ 16 h) → ejecución → daily (15 min, 3 preguntas) → incremento → review (producto) → retro (proceso). Sprint: timebox, de 1 semana a 1 mes, constante, sin cambios de objetivo, DoD. Cambio ≠ aclaración. DoD (todos) + criterios de aceptación (cada uno). 8 valores.

Rosa cambia de idea. Desarrollar = acuerdo; gestionar = mantenerlo. Depurar, control de cambios, ajuste de alcance, Kano, trazabilidad.

El croissant. Error → defecto → falla. Calidad = cumplir requisitos que reflejan las necesidades reales. Verificar = bien hecho; validar = lo correcto.

Después. Mantenimiento: correctivo, preventivo, adaptativo, aditivo, perfectivo, emergencia. SCM = identificar, controlar, registrar, verificar. Economía = ciencia de elegir.

. . .

Definición + para qué sirve + un ejemplo. Tres líneas. Si no sabes, en blanco.
Y si te quedas en blanco, piensa en Doña Rosa.